

Deep Learning 2025 Assignment 4

This is an **individual assignment** and its deadline is **Sunday, January 18, 2026, 22:00**. You must submit your solution electronically via the Absalon home page.

Diffusion models such as denoising diffusion probabilistic model (DDPM) and variational diffusion model (VDM) are formulated as hierarchical latent variable models. That is like a variational autoencoder with more than one layer of stochastic variables. In the VDM paper, it is shown that the hierarchical model has a well-defined infinite number of stochastic layers limit. This limit corresponds to defining the generative model through a stochastic differential equation. Alternatively, one may define the generative model using stochastic differential equations like in the closely related score and flow matching methods. One may also define the generative model using an ordinary differential equation as in the Neural ODE.

In this assignment, we will work with hierarchical variational autoencoders (exercise 1) and ordinary and stochastic differential equations — ODEs and SDEs (exercises 2 and 3).

1 Hierarchical variational autoencoder

The notebook `hierarchical_VAE.ipynb` implements a hierarchical VAE model. Your tasks are:

- 1.1. Briefly describe the code. How is dynamic binarization implemented, what is warm-up and what are the three defined networks' roles?
- 1.2. Run the code for a number of different values of T , the number of latent layers. $T = 1$ corresponds to a standard VAE. MNIST is a very simple dataset so going beyond $T = 5$ will not give a very big increase in performance. Do you observe an improved performance with increased T ?
- 1.3. You can get better performance by training longer and also use other tricks like fine-tuning with a decreased learning rate. This is not really what we are after here so just select your favorite T and run it for longer. Report the the number of warm-up and total epochs and the validation ELBO you achieved. Do you also observe visually better generated digits by running longer?
- 1.4. The importance weighted autoencoder (IWAE) bound is a tighter bound than the ELBO. It is more expensive to compute so we normally only use it when validating a trained model. It is also implemented in the code. You can find a few lines in the code commented out that you can use to calculate it. Report the IWAE for the longer run you did in the previous question.
- 1.5. Implement a parameterization of the hierarchical VAE closer in spirit to the SDE as discussed in the “From hierarchical VAE to differential equations”

part of the lecture slides:

$$\begin{aligned}\mu_{\theta,t}(\mathbf{z}) &= \mathbf{z} + \mathbf{f}(\mathbf{z}, t)\Delta t \\ \mu_{\phi,t}(\mathbf{x}, \mathbf{z}) &= \mathbf{z} + \mathbf{g}(\mathbf{x}, \mathbf{z}, t)\Delta t \\ \sigma_{\theta,t}(\mathbf{z}) = \sigma_{\phi,t}(\mathbf{z}) &= \sigma(\mathbf{z}, t)\sqrt{\Delta t}\end{aligned}$$

with $\Delta t = 1/(T - 1)$. You can use an LLM to help you do the needed modifications to the code. Do you see a change in performance using this less general parameterization? Report the performance for the configuration used in the two previous questions.

2 ODE

In this exercise, you will work with ordinary differential equations for density modeling in one dimension. A notebook `ODE.ipynb` implements a class to simulate training and validation data, models, model training and visualization. Your tasks will be about analyzing the approach and try different variations. Specifically, your tasks are:

- 2.1. **Background on the methodology:** State the generative model equations (see notebook) and log likelihood objective equations (see lecture slides). Just the equations nothing else.
- 2.2. **Analyze the program:** List the different parts of the notebook with brief descriptions of the function of each part (1-2 sentences) and its relation to the methodology. (Some functions and options in the code are not used. These of course should not be described.)
- 2.3. **Run training with Gaussian training data:** Run the program without changing anything. From what distribution and with what parameters are the training data generated? Include and describe the plot you obtain. Can the model learn the task?

In the program we generate data $x \sim p(x)$ from a distribution $p(x)$ that we can also evaluate exactly. We can therefore calculate the true expected log likelihood: $\frac{1}{n_{\text{test}}} \sum_{i=1}^{n_{\text{test}}} \log p(x^{(i)})$. This is the number that is returned from the dataset generator program. Compare this number with log likelihood that the final iteration gives. Include in the report the expected log likelihood that the dataset generator returns and the final log likelihood you achieve by running the program. Are the trained models close to the ideal value? Does this agree with the visualizations of the densities?

- 2.4. **Run training with mixture of Laplace training data:** Now change the program to run with mixture of Laplacians data instead. Do the same as above for this new dataset. Try to change the drift network. Can you improve the performance?

The simplest numerical integrator method is Euler (euler). The program also implements two Runge-Kutta methods (rk2 and rk4). Try to change the integration method. Does this change anything?

- 2.5. **Complexity:** The calculation of the log likelihood requires numerical integration of the ODE with T integration steps. ($T = 100$ in the program.) How will the complexity scale with T ? How will back propagation for parameter estimation scale with T ?

3 SDE

In this exercise, you will work with stochastic differential equations for density modeling in one dimension. A notebook `SDE.ipynb` implements a class to simulate training and validation data, models, model training and visualization. Your tasks are:

- 3.1. **Background on the methodology:** State the generative model (see notebook) and log likelihood lower bound (evidence lower bound, ELBO) objective equations (see lecture slides). Just the equations nothing else.
- 3.2. **Analyze the program:** List the different parts of the notebook with brief descriptions of the function of each part (1-2 sentences) and its relation to the methodology. No need to describe the code also found in the ODE notebook or code not used.
- 3.3. **Run training:** Run the program without changing anything. Include and describe the plot you have obtained. Also include in the report the expected log likelihood that the dataset generator returns and the final log likelihood you achieve by running the program.
- 3.4. **Change parameterization of the model components:** Try to change the network that specifies the variational distribution $q_t(z_t|x) = \mathcal{N}(z_t|\alpha(x,t), \beta^2(x,t))$ and generative model $f(z,t)$ and $\sigma(z,t)$. Some possible parameterizations are given in the notebook. Do you observe any changes to the behavior?
- 3.5. **Expected log likelihood:** Compare the ELBO values you get to the ODE case. Are they more or less the same? Does the SDE fit to the distributions look good visually? Why do you think that the SDE objective fluctuate more than the ODE objective? Hint: The SDE objective depends upon sampling random t -values.
- 3.6. **Complexity:** The ELBO objective is so-called simulation-free because it does not require integration of the differential equation. What does that mean for the complexity compared to the ODE approach?